

实验I Data Lab

1. bitXor

```
/*
 * bitXor — 只使用 ~ 和 & 计算  $x \oplus y$ 
 * 示例: bitXor(4, 5) = 1
 * 允许的运算符: ~ &
 * 最大运算符数: 14
 * 难度: 1
 */
int bitXor(int x, int y) {
    /* 保留只在 x 和 y 其中一个数中出现的位。 */
    return ~(~x & ~y) & ~(x & y);
}
```

2. tmin

```
/*
 * tmin — 返回最小的二进制补码整数
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 4
 * 难度: 1
 */
int tmin(void) {
    /* 最小补码整数只有符号位为 1。 */
    return 1 << 31;
}
```

3. isTmax

```
/*
 * isTmax — 如果 x 是最大的二进制补码整数则返回 1，否则返回 0
 * 允许的运算符: ! ~ & ^ | +
 * 最大运算符数: 10
 * 难度: 1
 */
int isTmax(int x) {
    /* 对 Tmax 而言, x + (x + 1) 等于 -1; 同时排除 x 等于 -1。 */
    int next = x + 1;
    return !((x + next) & !next);
}
```

4. allOddBits

```
/*
 * allOddBits — 如果字中所有奇数编号位均为 1，则返回 1
 * 位编号从最低有效位 0 到最高有效位 31
 * 示例: allOddBits(0xFFFFFFFF) = 0, allOddBits(0xAAAAAAAA) = 1
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 12
 * 难度: 2
 */
int allOddBits(int x) {
    /* 使用合法的单字节常量构造 0xAAAAAAAA。 */
    int mask = 0xAA | (0xAA << 8);
    mask = mask | (mask << 16);
    return !((x & mask) ^ mask);
}
```

5. negate

```
/*
 * negate — 返回 -x
 * 示例: negate(1) = -1
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 5
 * 难度: 2
 */
int negate(int x) {
    return ~x + 1;
}
```

6. isAsciiDigit

```
/*
 * isAsciiDigit — 如果 0x30 <= x <= 0x39, 则返回 1
 * 该范围对应字符 '0' 到 '9' 的 ASCII 编码
 * 示例: isAsciiDigit(0x35) = 1
 *         isAsciiDigit(0x3a) = 0
 *         isAsciiDigit(0x05) = 0
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 15
 * 难度: 3
 */
int isAsciiDigit(int x) {
    /* x - 0x30 与 0x39 - x 必须同时为非负数。 */
    int aboveLower = x + (~0x30 + 1);
    int belowUpper = 0x39 + (~x + 1);
    return !(aboveLower >> 31) & !(belowUpper >> 31);
}
```

7. conditional

```
/*
 * conditional — 实现与  $x ? y : z$  相同的效果
 * 示例: conditional(2, 4, 5) = 4
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 16
 * 难度: 3
 */
int conditional(int x, int y, int z) {
    /* 将 x 的真假值扩展为全 0 或全 1 的掩码。 */
    int truth = !!x;
    int mask = ~truth + 1;
    return (mask & y) | (~mask & z);
}
```

8. isLessOrEqual

```
/*
 * isLessOrEqual — 如果  $x \leq y$  则返回 1, 否则返回 0
 * 示例: isLessOrEqual(4, 5) = 1
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 24
 * 难度: 3
 */
int isLessOrEqual(int x, int y) {
    /* 异号时不会发生减法溢出; 同号时检查  $y - x$  的符号。 */
    int xNegative = (x >> 31) & 1;
    int yNegative = (y >> 31) & 1;
    int signsDiffer = xNegative ^ yNegative;
    int difference = y + (~x + 1);
    int differenceNonnegative = !(difference >> 31);
    return (signsDiffer & xNegative) | ((!signsDiffer) & differenceNonnegative);
}
```

```
}
```

9. logicalNeg

```
/*
 * logicalNeg — 不使用 ! 运算符实现逻辑非
 * 示例: logicalNeg(3) = 0, logicalNeg(0) = 1
 * 允许的运算符: ~ & ^ | + << >>
 * 最大运算符数: 12
 * 难度: 4
 */
int logicalNeg(int x) {
    /* 对任意非零 x, x 和 -x 中至少有一个数的符号位为 1。 */
    return ((x | (~x + 1)) >> 31) + 1;
}
```

10. howManyBits

```
/*
 * howManyBits — 返回使用二进制补码表示 x 所需的最少位数
 * 示例: howManyBits(12) = 5
 *        howManyBits(298) = 10
 *        howManyBits(-5) = 4
 *        howManyBits(0) = 1
 *        howManyBits(-1) = 1
 *        howManyBits(0x80000000) = 32
 * 允许的运算符: ! ~ & ^ | + << >>
 * 最大运算符数: 90
 * 难度: 4
 */
int howManyBits(int x) {
    /* 先将负数规格化, 再使用二分查找定位最高的 1 位。 */
    int b16, b8, b4, b2, b1;
```

```

x = x ^ (x >> 31);
b16 = !! (x >> 16) << 4;
x = x >> b16;
b8 = !! (x >> 8) << 3;
x = x >> b8;
b4 = !! (x >> 4) << 2;
x = x >> b4;
b2 = !! (x >> 2) << 1;
x = x >> b2;
b1 = !! (x >> 1);
x = x >> b1;
return b16 + b8 + b4 + b2 + b1 + x + 1;
}

```

11. floatScale2

```

/*
 * floatScale2 — 返回浮点数表达式 2*f 的位级等价值
 * 参数和返回值均以 unsigned 传递，但应解释为单精度浮点数的位表示。
 * 当参数为 NaN 时，直接返回原参数。
 * 允许的运算符：任意 int/unsigned 运算，包括 ||、&&、if 和 while
 * 最大运算符数：30
 * 难度：4
 */
unsigned floatScale2(unsigned uf) {
    unsigned sign = uf & 0x80000000u;
    unsigned exponent = uf & 0x7F800000u;

    if (exponent == 0x7F800000u) {
        return uf;
    }

    if (exponent == 0) {
        return sign | ((uf & 0x7FFFFFFFu) << 1);
    }
}

```

```

    exponent = exponent + 0x00800000u;

    if (exponent == 0x7F800000u) {
        return sign | exponent;
    }

    return sign | exponent | (uf & 0x007FFFFFFu);
}

```

12. floatFloat2Int

```

/*
 * floatFloat2Int — 返回浮点表达式 (int)f 的位级等价值
 * 参数以 unsigned 传递，但应解释为单精度浮点数的位表示。
 * 对任何超出范围的值（包括 NaN 和无穷）返回 0x80000000u。
 * 允许的运算符：任意 int/unsigned 运算，包括 ||、&&、if 和 while
 * 最大运算符数：30
 * 难度：4
 */
int floatFloat2Int(unsigned uf) {
    unsigned sign = uf >> 31;
    unsigned exponent = (uf >> 23) & 0xFF;
    unsigned fraction = (uf & 0x007FFFFFFF) | 0x00800000;
    int power = exponent + (~127 + 1);
    unsigned value;

    if (exponent == 0xFF || power > 30) {
        return 0x80000000u;
    }

    if (power < 0) {
        return 0;
    }

    if (power > 23) {
        value = fraction << (power - 23);
    } else {
        value = fraction >> (23 - power);
    }
}

```

```

    }

    if (sign) {
        return ~value + 1;
    }

    return value;
}

```

13. floatPower2

```

/*
 * floatPower2 — 对任意 32 位整数 x，返回表达式 2.0^x 的位级等价值
 *
 * 返回的 unsigned 应与单精度浮点数 2.0^x 具有完全相同的位表示。
 * 如果结果小到无法表示为非规格化数，则返回 0；如果结果过大，
 * 则返回正无穷。
 *
 * 允许的运算符：任意 int/unsigned 运算，包括 ||、&&、if 和 while
 * 最大运算符数：30
 * 难度：4
 */
unsigned floatPower2(int x) {
    if (x < -149) {
        return 0;
    }

    if (x < -126) {
        return 1u << (x + 149);
    }

    if (x <= 127) {
        return (x + 127) << 23;
    }

    return 0x7F800000u;
}

```

运行结果

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

root@DESKTOP-Q58LPNS:~/csapp/datalab# ./driver.pl
2. Compiling and running './btest -g' to determine correctness score.
gcc -O -Wall -lm -o btest bits.c btest.c decl.c tests.c
btest.c: In function 'test_function':
btest.c:334:23: warning: 'arg_test_range' may be used uninitialized [-Wmaybe-uninitialized]
 334 |     if (arg_test_range[2] < 1)
      |           ~~~~~^~~~~
btest.c:299:9: note: 'arg_test_range' declared here
 299 |     int arg_test_range[3]; /* test range for each argument */
      |           ~~~~~
3. Running './dlc -Z' to identify operator count violations.

4. Compiling and running './btest -g -r 2' to determine performance score.
gcc -O -Wall -lm -o btest bits.c btest.c decl.c tests.c
btest.c: In function 'test_function':
btest.c:334:23: warning: 'arg_test_range' may be used uninitialized [-Wmaybe-uninitialized]
 334 |     if (arg_test_range[2] < 1)
      |           ~~~~~^~~~~
btest.c:299:9: note: 'arg_test_range' declared here
 299 |     int arg_test_range[3]; /* test range for each argument */
      |           ~~~~~

5. Running './dlc -e' to get operator count of each function.

Correctness Results      Perf Results
Points  Rating  Errors  Points  Ops    Puzzle
1       1       0       2       7     bitXor
1       1       0       2       1     tmin
1       1       0       2       7     isTmax
2       2       0       2       7     allOddBits
2       2       0       2       2     negate
3       3       0       2      11     isAsciiDigit
```